Classes

CS101 Introduction to Intermediate Programming CPU Spring 2026

Table of Contents

1. Objectives

Covered:

1. [X] Procedural vs. Object-Oriented Programming
2. [X] Defining classes and objects
3. [X] Private and public access control
4. [X] Member functions
5. [X] Constructors and destructors
6. [X] Constructor and operator overloading
7. [X] Separation of interface and implementation
8. [X] Copy constructors

Other topics:

1. [ ] Arrays of Objects
2. [ ] Simulating dice (case study)
3. [ ] Unified Modeling Language (UML)
4. [ ] Instance and static members
5. [ ] Friends of Classes
6. [ ] Protected data members
7. [ ] Memberwise Assignment
8. [ ] Object Conversion
9. [ ] Aggregation
10. [ ] Class collaborations (case study)
11. [ ] Game simulation (case study)
12. [ ] Rvalue References and Move Semantics
13. [ ] Inheritance (derivative classes)
14. [ ] Polymorphism (share user interface at runtime)
15. [ ] Virtual Functions (overriding member functions at runtime)

2. Procedural vs. object-oriented programming (OOP) paradigms

- Most disciplines that grow over time are not just cobbled together: People identify patterns that are successful, explore and pursue them, and finally turn them into standards (until they fail).
- These patterns are also called "paradigms" (παραδείγματα). Two important programming paradigms are procedural and object-oriented programming.

- What does **procedural** mean? What does it achieve?

  > Procedural programming is centered on creating procedures or functions. Its main purpose is modularization.

- What is **object orientation**? What does it achieve?

  Object-orientation means that everything is an object. Each object has its own state and behavior. Computation happens by message exchange between objects, like a conversation between agents. It enables **encapsulation** (of data), **inheritance** (of objects), and **polymorphism** (at compile-time or at run-time).

3. Example: A light switch as message exchange between objects

- Example 1: A light switch.

```
#include <iostream>
using namespace std;

class Light {
public:
  void turnOn() {cout << "Light is on.\n";}
  void turnOff() {cout << "Light is off.\n";}
};

class Switch {
  Light *light; // knows where to send the message
public:
  Switch(Light *l) : light(l) {}; // flips the switch
  void flip(bool on) {
    if (on)
      light->turnOn(); // sends 'turnOn' message
    else
      light->turnOff(); // sends 'turnOff' message
  }
};

int main()
  {
    Light lamp; // create a lamp
    Switch s(&lamp); // create a switch for the lamp
    s.flip(true); // throw the switch
    s.flip(false); // throw the switch
    return 0;
    }
```

- Explanation:

  The `Switch` does not directly manipulate the `Light` data - it just sends messages `turnOn` and `turnOff`. Each object controls its own behavior. This models **message passing** between objects instead of function calls manipulating shared data.

- **Challenge:** What would the procedural version of this program look like? (Bonus problem)

4. Example: Procedural vs. object-oriented adding of objects

- What is **object-oriented** programming?

  Object-oriented programming is centered on creating and manipulating objects in a safe way: data are encapsulated can only be operated on by member functions.

- Example using an `add` function in procedural style:

1. You may want to add integers.
2. You may want to add floating-point numbers.
3. You may want to "add" (concatenate) strings.

These are three different procedures. In C++, they can be **overloaded** so that each function is called `add` but which one is used depends on its arguments.

- Implementation:

```
#include <iostream>
using namespace std;

// Integer addition
int add(int a, int b)
{
  return (a+b);
}

// Floating-point addition
double add(double a, double b)
{
  return (a+b);
}

// String concatenation
string add(std::string a, std::string b)
{
  return (a+b);
}
// The main program calls each function
int main()
{
  cout << add(5,7) << endl;
  cout << add(5.5,6.5) << endl;
  cout << add("five","seven") << endl;
  return 0;
}
```

- In the OOP style, a `class` named `Adder` has member functions that take care of both argument types:

```
#include <iostream>
using namespace std;

// The Adder class has 'public' methods (no data)
class Adder {
public:
  int add(int a, int b) // integer addition
  {
    return (a+b);
  }
  double add(double a, double b) // double addition
  {
    return (a+b);
  }
  string add(std::string a, std::string b)
  {
    return (a+b);
  }
};
// The functions are called on the Adder object A
int main()
```

```
{
  Adder A;
  cout << A.add(5,7) << endl;
  cout << A.add(5.5,6.5) << endl;
  cout << A.add("five","seven") << endl;
  return 0;
}
```

```
12
12
fiveseven
```

- Problems with Procedural Style
    - All `add` functions live in **global scope** — no logical grouping.
    - Namespace pollution: more functions → more name conflicts.
    - Maintenance gets harder as more types/behaviors are added.
    - Can't easily associate behavior with **state** (no objects).
- Advantage of OOP
    - All `add` methods are grouped in **one class**: `Adder`.
    - Code is modular and encapsulated — easier to reason about.
    - Behavior can be extended by adding methods like `reset` or `showState`.
    - Can associate behavior with **object state** like `lastResult` or `count`.
- Criticisms of OOP
    - Can incur significant overhead through deep class hierarchies.
    - Misused inheritance (derived classes) can lead to tangled code.
    - OOP adds needless structure to simple data logic.
    - Encapsulation as an illusion: data can leak through getters.
    - Tight coupling: Classes can become too interdependent.
    - Difficult testing: Objects with state can be hard to test.
- Why do "objects" and their "state" matter?
    - State (data) and behavior (methods) are separate and together.
    - Objects mimic real identities, e.g. a `BankAccount` has **data** like a `balance` and **methods** (behavior) like `deposit` or `withdraw`.
    - Objects can be reused and evolved by extending classes.
    - In UI and games, stateful interactions are useful (Game on/off).

5. An everyday "data encapsulation" example: The car

- What's part of the car's "user interface"? (What can you see/use?)

  To use a car (as driver)
    - Ignition switch
    - Steering wheel
    - Gas pedal
    - Brake pedal
    - Gear shift

    - Dashboard display

- What's part of the car's "internal implementation"? (What's hidden?)

    The mechanisms hidden from the user to be able to use it.

    - Steering column
    - Fuel injection
    - Transmission system
    - Brake hydraulics
    - Most things outside the driver's field of vision

6. Classes and objects

- What is a `class`? A `class` is three things: **keyword**, **code**, and **concept**.

- As a **keyword**:

    As a **keyword** a `class` is a specifier for a user-defined **type** that you can use to cast data in any (digital) form you like. You can make your own categories.

    So for example, the `Light` class allowed us to define a `lamp`. As an object, a `lamp` is like an integer or a string: memory + action.

- As **code**:

    As code, a `class` specifies (aka encapsulates) the attributes (properties, data) and behavior (functions, methods) that an **object** of that class may have.

```
// The Rectangle class objects have width and length
// You can set and get these properties for any Rectangle
class Rectangle { // class name

private: // member data/attributes/properties
  double width;
  double length;

public:  // member functions/methods/behavior
  void setWidth(double);
  void setLength(double);
  double getWidth();
  double getlength();
  double getArea();
};
```

- As a **concept:**

    As a concept, a `class` it is the **description** of an object, like a blueprint:

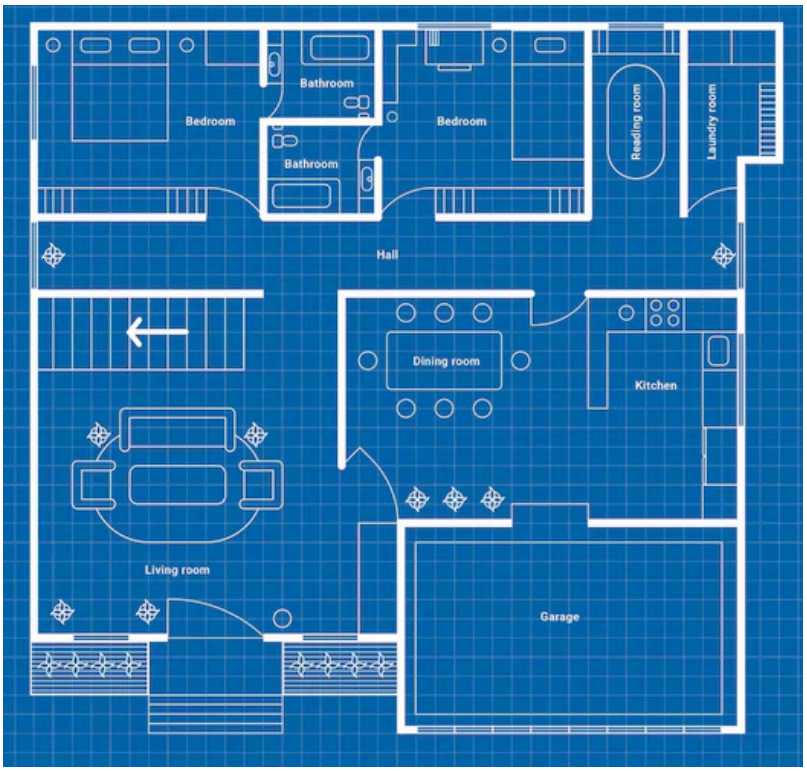


Figure 1: Blueprint of a house.

- The blueprint can be used to build any number of objects, or **instances**.

Figure 2: House objects build with a house blueprint.

- Example: What could you say about the `Insect` class and the two objects shown in the figure?

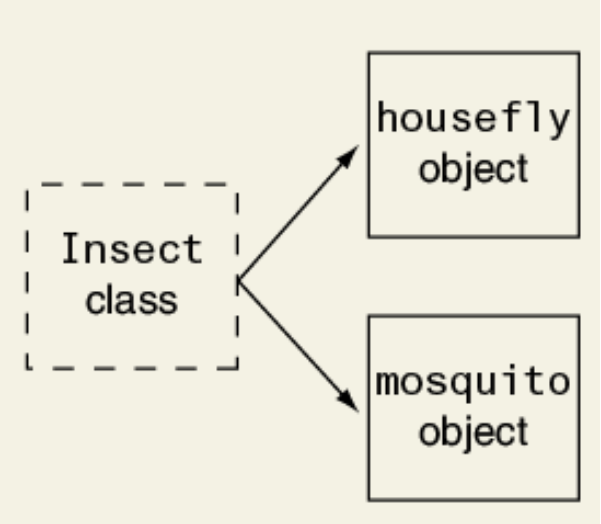


Figure 3: An Insect class and two of its instances.

Answer:

> The `Insect` class could describe attributes and abilities of insects. The `housefly` object and the `mosquito` object are **instantiated** from `Insect`. Instances have all of the attributes and abilities described in the `Insect` class.
>
> A lot of the code written for `Insect` applications can now be shared by applications for housefly and mosquito insects. When a `housefly` or `mosquito` are created, they get their own area in memory.
>
> This makes conceptual sense: some properties/behavior is not shared between these - mosquitos and houseflies have different mouthparts, mosquitos sting and suck blood, houseflies feed by sponging.

7. Glossary - check your understanding of basic terminology and examples!

Term	Definition
Procedural Prog.	Programming with procedures and global data
OOP	Objects encapsulate data & logic
Encapsulation	Hide internal data; access via member functions
State/attribute	Data stored in an object

Term	Definition
Behavior	Actions defined by an object's methods
Car UI	Controls used (wheel, pedals, dashboard)
Car Implementation	Internal mechanisms (steering, fuel, brakes)
class (keyword)	Declares a user-defined type
class (code)	Contains attributes and member functions
class (concept)	Blueprint to create instances of an object
Object/Instance	Realized form of a class in memory
Insect (class)	Class describing common insect features
housefly/mosquito	Instances of Insect class

8. Using the `string` class

- You've already used C++ classes like `vector` and `string`.
- To use the `string` class you need to `#include <string>`. Now you can define `string` objects, and use string member functions.

- Example:

```
#include <iostream>
#include <string>
using namespace std;

int main()
{
  string cityName;                    // create a string object
  cityName = "Batesville";            // assign a string to it
  cout << cityName << endl;           // print string content
  cout << cityName.length() << endl; // get string size in bytes
  cout << cityName.append(", AR");   // append another string

  return 0;
}
```

```
Batesville
10
Batesville, AR
```

- What about the time complexity of the `length` and `append` functions?

Operation	Time	Notes
`std::string::length`	O(1)	Length stored internally at compile-time
`std::string::append`	O(1)	Places new characters in allocated space

In terms of **time**, both of these are as efficient as can be: When a `string` is created, its `length` is stored internally already. The `append` operation works similar to `vector::push_back` (which is O(1)).

9. Introduction to classes (`BankAccount`)

Let's use a **bank account** as an example (since everyone has got one).

1. What are the computing **procedures** to run a **bank account**?

    - Getting the balance (getter/accessor function)
    - Depositing an amount (setter/mutator function)
    - Withdrawing an amount (setter/mutator function)

    All of these are going to be "`public` member functions": This means that they are available to any bank account object. It's what a client can do with a bank account.

2. What are the **data** of a bank account?

    - Current balance
    - Amount to be deposited (function argument)
    - Amount to be withdrawn (function argument)

    The `balance` is going to be "`private` member data": We don't want anyone manipulating the `balance` without our designed functions.

3. What does designing a `BankAccount` `class` mean?

    - Writing `BankAccount` procedures that manage the data flow.
    - Creating ("instantiating") objects in `main`.
    - Using `BankAccount` objects in `main`.

4. What's an **object**?

    An object is a resource used by the program to complete its task, no difference from other resources, like variables, that are in memory.

    To the `class`, an object is an instance of what the `class` represents - for example a `lamp` is a `Light` object, a `savingsAccount` is a `BankAccount` object, etc.

5. Example of an object: A savings account. What does it have?

    A savings account is a `BankAccount` object:

    1. The object has data (balance, account number etc.)
    2. It has operations (deposit, withdrawal, balance)
    3. We can interact with it: "Tell me the balance", "Withdraw $100".

6. What's a constructor? Example?

    A constructor constructs objects of the chosen class. In the case of `BankAccount`, our constructor creates a `BankAccount` with a starting balance.

10. OOP workflow and class template

- How to do it in practice:

    1. Write a class: data and methods = description/blueprint.
    2. Create instances of the class = objects.
    3. Use class methods on class data = OOP

- General template:

```
class ClassName {

  private:
     // data members (attributes)

  public:
     // member functions (methods)
};
```

- Member functions can be defined **inline** (inside the `class` template), or externally. Then they have to be brought into **scope** with the scope operator `::`

- Example: Code along using OneCompiler.Com. Insert this code beetween the `namespace` statement and the `main` program, like a function. Note: All of the member functions are defined **inline**.

```
class BankAccount
{
private:
  double balance;

public:
  BankAccount(double startingBal) // constructor
  { balance = startingBal; }

  void deposit(double amount)  // mutator/setter
  { balance += amount; }

  void withdraw(double amount) // mutator/setter
  { balance -= amount; }

  double getBalance() const // accessor/geetter
  { return balance; }

};
```

11. Member functions and member data encapsulation

- There are some similarities between defining/using functions (procedures) and classes.

- What if we do not define a member function inside the `class` definition?

    If the first function (constructor) `BankAccount` for example, would be defined externally, the `class` declaration would only contain its prototype while the function definition/implementation would be elsewhere.

- The compiler has to be told that it is part of the `class`:

```
BankAccount::BankAccount(double startingBal)
{
  balance = startingBal;
}
```

- The main differences to the **inline** definition:

    1. The function parameter `startingBal` is no longer optional.
    2. The function needs to be brought into the scope of the class it talks to, with the prefix `BankAccount::` - the `class` name followed by the scope resolution operator `::`
    3. The function must follow the `class` declaration, and usually before the `main` function.
    4. In the case of multiple classes, the `class` declaration resides in its own header file `BankAccount.h`, while the member function definitions are in a separate file `BankAccount.cpp`, which is compiled separately and linked to the main program as an `.o` object file.

- What's going on conceptually here?

    This is an example of **Separation** of **Interface** and **Implementation** - it mirrors the separation of function **prototype** and function **definition** is the same thing in procedural programming.

- We made `BankAccount::balance` `private` to the `class` so that only our member functions can directly access it. The variable `balance` is **encapsulated**.
- The **constructor**, `BankAccount(double startingBal)` requires from us that any bank account can only be created with a starting balance.
- `deposit` and `withdraw` are **mutator** or **setter** functions. They allow us to change the data members of the class. They usually have arguments to import data (like `amount`) but the often are `void` because they're not getting any values for us.
- Functions like `getBalance` are **accessor** or **getter** functions. They allow us to access data members of the class. Since they don't change any values, they are usually declared `const`, and they often don't have any arguments but they have `return` types (like `double`).

12. Using a class in a `main` function

- We must manage the data flow between the `class` definition and the `main` program, where `class` objects are created and used.
- Example with `BankAccount`:
    1. Create a `BankAccount` named `acct` with 100 starting balance.
    2. Show the current balance.
    3. Deposit 10 and show balance.
    4. Withdraw 50 and show balance.

- Code: Use your previous definition of `BankAccount` and change `main`.

```
#include <iostream>
using namespace std;

class BankAccount
{
private:
  double balance;

public:
```

```
  BankAccount(double startingBal)
  { balance = startingBal; }

  void deposit(double amount)
  { balance += amount; }

  void withdraw(double amount)
  { balance -= amount; }

  double getBalance() const
  { return balance; }

};

int main()
{
  BankAccount acct(100.0);
  cout << "Current balance: " << acct.getBalance() << endl;
  acct.deposit(10.0);
  cout << "Current balance: " << acct.getBalance() << endl;
  acct.withdraw(50.0);
  cout << "Current balance: " << acct.getBalance() << endl;
  return 0;
}
```

```
Current balance: 100
Current balance: 110
Current balance: 60
```

13. PRACTICE Separate interface and implementation

- Using the `BankAccount` class provided in the starter code, separate the class interface and the implementation by splitting the code into:
  1. `class` declaration with member data and member function prototypes,
  2. definitions of the member function prototypes.
  3. When you're done, create a `BankAccount` named `account` with a starting balance of 200, deposit 300, withdraw 100, and print the balance each time.

- Sample output:

```
Account starting balance: $200.
Deposited $300. New balance: $500.
Withdrew $100. New balance: $400.
```

- Starter code: tinyurl.com/bank-account-test

```
// The BankAccount class has a balance.  It is created with a starting
// balance. You can deposit or withdraw funds. You can get a balance
// printout.
class BankAccount
{
private:
  double balance;

public:
  BankAccount(double startingBal)
  { balance = startingBal; }
```

```
  void deposit(double amount)
  { balance += amount; }

  void withdraw(double amount)
  { balance -= amount; }

  double getBalance() const
  { return balance; }

};
```

- Solution video https://youtu.be/CyJvZ7nB_IU)

  Solution code: https://tinyurl.com/bank-account-separation

```
#include <iostream>
using namespace std;

// The BankAccount class has a balance.
// It is created with a starting balance.
// You can deposit or withdraw funds.
// You can get a balance printout.
// Author: Marcus Birkenkrahe (pledged)
// Class declaration
class BankAccount
{
private:
  double balance;

public:
  // Member function prototypes
  BankAccount(double starting_balance);
  void deposit(double amount);
  void withdraw(double amount);
  double getBalance() const;
};

// Class member function definitions
BankAccount::BankAccount(double startingBal)
{ balance = startingBal; }

void BankAccount::deposit(double amount)
{ balance += amount; }

void BankAccount::withdraw(double amount)
{ balance -= amount; }

double BankAccount::getBalance() const
{ return balance; }

// Main function
int main()
{
  // Create BankAccount
  BankAccount account(200.);
  cout << "Account starting balance: $" << account.getBalance() << ".\n";
  // Deposit amount
  account.deposit(300.);
  cout << "Deposited $300. New balance: $" << account.getBalance() << ".\n";
  // Withdraw amount
  account.withdraw(100.);
  cout << "Withdrawn $100. New balance: $" << account.getBalance() << ".\n";
```

```
  return 0;
}
```

14. Example: A `Rectangle` class

- Which data members should a `Rectangle` class have? Which types?

  A rectangle, as a geometric object, is fully defined by its **width** and by its **length**. These should be `double` precision data.

- Which member functions should a `Rectangle` class have?

  Every class needs **getter** (accessor) functions to extract the `private` data, and **setter** (mutator) functions to pass values to the `private` data. A default constructor is provided but it is better to code it explicitly (otherwise the data values are undefined).

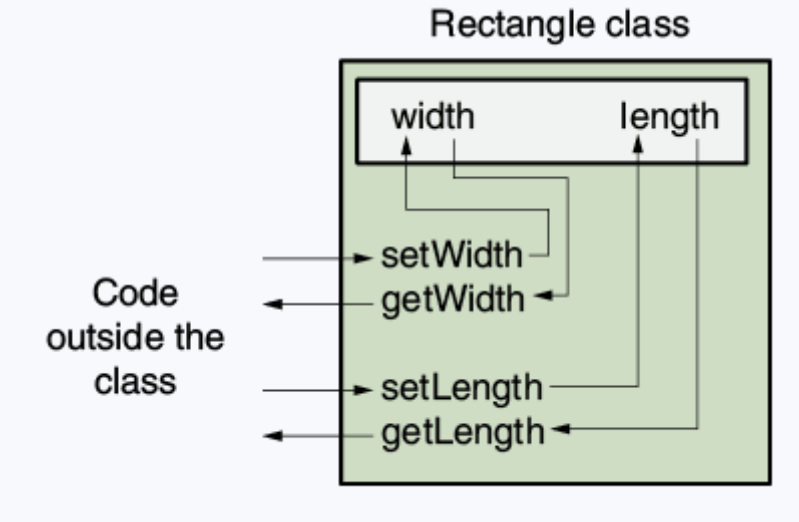
  Then there are things you can **do with the data** to compute derived quantities. For the rectangle, these are the **area** and the **perimeter**.

  - Getters: `getWidth` and `getLength`; optional: `getArea`, `getPerimeter`
  - Setters: `setWidth` and `setLength`
  - Optional but better: constructor, `Rectangle`.

- What `return` types and arguments should the member functions have?

  - The getter functions `return` the data so they are `double` functions, but they don't change the data so they can be `const`.
  - The `private` data members are `double`, so all setters must have `double` parameters and need not `return` anything so they are `void` functions.

- In OOP, an object protects its important data by making them `private`, and provides a `public` interface to access the data.



15. PRACTICE Create a `Rectangle` class

- Using the starter code below, **write a declaration** for the `Rectangle` class that includes the `width` and `length` data members, and the member functions `getWidth`, `getLength`, `setWidth`, and `setLength`.

  Write all member functions as **inline functions** (definitions in the `class` declaration). Remember to **run your code** as soon as you finished a statement for syntax checks - as soon.

  Test this class using the `main` function:

  1. Create a `Rectangle` object `r` with width 4 and length 5.
  2. Print `width` and `length` of the object.

- Sample output:

```
Width = 4
Length = 5
```

- Starter code: tinyurl.com/rectangle-class-starter

```
// Declare and test a Rectangle class
// Header files

// Namespace declaration

// Class declaration

// private member data
// rectangle width
// rectangle length
// public member functions

// set rectangle width
// set rectangle length
// get rectangle width
// get rectangle length

// Main function

// create a Rectangle object r
// set object width to 4
// set object length to 5
// show object width
// show object length
```

- Solution: https://tinyurl.com/rectangle-class-solution

```
// Declare and test a Rectangle class
// Header files
#include <iostream>
// Namespace declaration
using namespace std;

// Class declaration
class Rectangle {
  // private member data
private:
  double width; // rectangle width
  double length;  // rectangle length
  // public member functions
```

```
public:
  void setWidth (double w) { width = w; } // set rectangle width
  void setLength (double l) { length = l; } // set rectangle length
  double getWidth() const { return width; } // get rectangle width
  double getLength() const { return length; } // get rectangle length
};
// Main function
int main()
{
  Rectangle r; // create a Rectangle object r
  r.setWidth(4.);  // set object width to 4
  r.setLength(5.);  // set object length to 5
  cout << "Width = " << r.getWidth() << endl; // show object width
  cout << "Length = " << r.getLength() << endl; // show object length
  return 0;
}
```

16. TODO Bonus assignment: Separate interface and implementation for `Rectangle`

- Modify the declaration for the `Rectangle` class to separate the `class` declaration with prototypes and the member function implementation.
- Each function should have its own short header explaining what it does. Example for two member functions of the `BankAccount` class:

```
//*************************************************************
// withdraw subtracts the amount from the private member balance *
//*************************************************************
void BankAccount::withdraw(double amount)
{ balance -= amount; }

//*************************************************************
// getBalance returns the value in the private member balance    *
//*************************************************************
double BankAccount::getBalance() const
{ return balance; }
```

- After the separation, the test in `main` should still work and generate the same output:

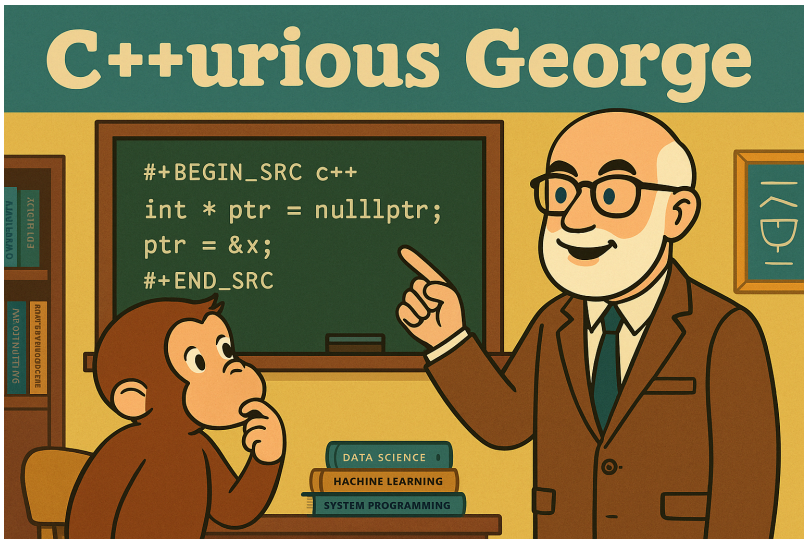
```
Account starting balance: $200.
Deposited $300. New balance: $500.
Withdrawn $100. New balance: $400.
```

- Submit the onecompiler.com URL to your solution in Canvas.
- Solution:

```
// Declare and test a Rectangle class
// Header files
#include <iostream>
// Namespace declaration
using namespace std;

// Class declaration
class Rectangle {
```

```
  // private member data
private:
  double width; // rectangle width
  double length;  // rectangle length
  // public member functions
public:
  void setWidth (double );  // set rectangle width
  void setLength (double ); // set rectangle length
  double getWidth() const;  // get rectangle width
  double getLength() const; // get rectangle length
};

// Main function
int main()
{
  Rectangle r; // create a Rectangle object r
  r.setWidth(4.);  // set object width to 4
  r.setLength(5.);  // set object length to 5
  cout << "Width = " << r.getWidth() << endl; // show object width
  cout << "Length = " << r.getLength() << endl; // show object length
  return 0;
}

// Member function definitions
//*************************************************************
// setWidth assigns its argument to the private member width     *
//*************************************************************
void Rectangle::setWidth (double w)
{
  width = w;
}
//*************************************************************
// setLength assigns its argument to the private member length   *
//*************************************************************
void Rectangle::setLength (double l)
{
  length = l;
}
//*************************************************************
// getWidth returns the value in the private member width        *
//*************************************************************
double Rectangle::getWidth() const
{
  return width;
}
//*************************************************************
// getLength returns the value in the private member length      *
//*************************************************************
double Rectangle::getLength() const
{
  return length;
}
```

17. C++urious George: Object choices, default constructions, multiple value returns



- What would be an **object** to start with for a bank program?

    1. A bank account.
    2. A bank account holder.
    3. A bank.

    Of these three, the bank account is the best candidate to start with because it's got range (of operations), and is fairly self-contained. The account holder data and the bank are attached to it, and they exist at different levels of abstraction.

- What about when we want to draw a **rectangle**? Which objects could be relevant?

    1. A rectangle.
    2. A line.
    3. A point.
    4. A length.
    5. A width.
    6. A person who draws the rectangle.

- Is a default constructor always provided without declaration?

    Yes but the data member values are not defined until they are initialized, either with a setter function or with a constructor.

```
class Rectangle {
private:
  double width;
  double length;
public:
  double getWidth() const {return width;}
  double getLength() const {return length;}
};
int main()
{
  Rectangle ra;
  cout << ra.getWidth() << endl;
  cout << ra.getLength() << endl;
```

```
  return 0;
}
```

```
6.45668e-310
6.45668e-310
```

- Why not just define a member function `getData` and `return` all member data with one function?

  > You can only `return` a single value (or object) at a time. To `return` multiple member values, you must used a composite type (like a `class` or a `struct`), and the `class` construction is a lot more difficult!

  Code example:

```
struct RectangleData {
  double width;
  double length;
};

class Rectangle {
private:
  double width;
  double length;
public:
  // constructor
  Rectangle(double w, double l)
    : width(w), length(l) {}

  // getter function
  RectangleData getData() const {
    RectangleData data;
    data.width = width;
    data.length = length;
    return data;
  }
};
int main()
{
  Rectangle ra(4.0, 5.0);
  RectangleData d=ra.getData();
  cout << "Width: " << d.width << "\nLength: " << d.length << endl;
  return 0;
}
```

```
Width: 4
Length: 5
```

18. Review questions

1. `private` and `public` are known as what?
   - [ ] Class declarators
   - [ ] Assignment delimiters
   - [X] Access specifiers
   - [ ] Member accessors
2. Which symbol delimits `private` and `public`?

- [ ] Curly braces {}
- [ ]
- [ ] A semicolon ;
- [X] A colon :

3. Where are class data members and class member functions usually located in the `class` declaration?

    - Data members are `private` so that they are protected.
    - Member functions are `public` so that they can be used to access the data.

4. What does the keyword `const` written after the parentheses at the end of a member function declaration mean? As in this example:

    ```
    int getLength() const;
    ```

    The keyword `const` written after the parentheses of a member function declaration means that the function will not change any data stored in the calling object.

5. If I create a `Rectangle` object named `ra`, how do I call the `setWidth` function to set the `width` of the rectangle to the value 5?

    ```
    ra.setWidth(5.); // to set
    ```

6. What is the prescribed order for `private` and `public` members within the `class` declaration?

    `private` and `public` sections of the declaration can be written in any order.

7. Assume that `RetailItem` is the name of a `class`, and that the class has a `void` member function `setPrice` which accepts a `double` argument. Which of these show the correct use of the scope resolution operator?
    - [ ] `void setPrice (double p)`
    - [X] `void RetailItem::setPrice (double p)`
    - [ ] `void setPrice (double p)::RetailItem`
    - [ ] `RetailItem::void setPrice (double p)`

8. Which message do you expect if you use an externally defined member function if you leave the class name and scope resolution operator out of the functions's header? How do you explain it?

    An error message (compiler error): The function cannot be found within the scope because the function has not become a member of the class.

9. What is an accessor, what is a mutator in OOP?

    - An accessor, or getter, is a class member function that gets a velue from a class variable but does not change it (`const`).
    - A mutator, or setter, is a class member function that changes the value of a class variable but does not get it (`void`).

10. When you mark a member function as `const`, the `const` keywoard must appear in:
    - [ ] The function header of all member functions.
    - [ ] The function header but not the declaration.
    - [ ] The declaration only.
    - [ ] Both the declaration and the function header.

19. Defining an Instance of a Class (VideoNote 13.2)

- How do we create a `BankAccount` object?

  > Similar to type definitions `int x;`, a `class` is a user-defined type, so a `class` instance (an object) is created like this: `Class x;` This invokes the default constructor.

- How many instances of a `class` can we create?

  > Any number as long as there's memory available to us.

- How are the data members (e.g. `balance` of `BankAccount`) of different instances, linked? Will changing one change the other? For example:

```
BankAccount savingsAccount(2500.0);
BankAccount checkingAccount(500.0);
```

Answer:

> The data are not linked at all: Each object, `savingsAccount` and `checkingAccount` has its own copy of `balance`, stored independently in memory. Changing one will not change the other.

```
class BankAccount
{
private:
  double balance;

public:
  BankAccount(double startingBal)
  { balance = startingBal; }

  void deposit(double amount)
  { balance += amount; }

  void withdraw(double amount)
  { balance -= amount; }

  double getBalance() const
  { return balance; }

};

BankAccount savingsAccount(2500.0);
cout << "Savings: " << savingsAccount.getBalance() << endl;
BankAccount checkingAccount(500.0);
cout << "Checking: " << checkingAccount.getBalance();
```

```
Savings: 2500
Checking: 500
```

20. TODO Home assignment: Complete the `Rectangle` class

- **Problem:** Use the `Rectangle` class definition in the starter code. Add the member functions `setWidth`, `setLength`, `getWidth`, and `getLength` as external member functions (outside of the `class` definition). Add a new member function `getArea` that computes the area of the rectangle. Test the member function with two `Rectangle` objects called `box1` (`12.7` by `14.5`), and `box2` (`8.5` by `29.3`) by comparing their area.

- **Sample output:**

```
Box 1: 12.7 x 14.5
Box 2: 8.5 x 29.3
Box 1 is smaller.
```

- **Starter code:** tinyurl.com/rectangle-assignment - `Rectangle` class code in one file. Code compiles without output.

```
// Declare and test a Rectangle class
// Header files
#include <iostream>
// Namespace declaration
using namespace std;

// Class declaration
class Rectangle {
  // private member data
private:
  double width; // rectangle width
  double length;  // rectangle length
  // public member functions
public:
  void setWidth (double );  // set rectangle width
  void setLength (double ); // set rectangle length
  double getWidth() const;  // get rectangle width
  double getLength() const; // get rectangle length
};

// Member function definitions
//*************************************************************
// setWidth assigns its argument to the private member width     *
//*************************************************************

//*************************************************************
// setLength assigns its argument to the private member length   *
//*************************************************************

//*************************************************************
// getWidth returns the value in the private member width        *
//*************************************************************

//*************************************************************
// getLength returns the value in the private member length      *
//*************************************************************
```

- **Extension:** We will work in class with separated header and implementation files using Google Cloud shell.

- Solution:

```
// Declare and test a Rectangle class
// Header files
```

```cpp
#include <iostream>
// Namespace declaration
using namespace std;

// Class declaration
class Rectangle {
  // private member data
private:
  double width; //rectangle width
  double length;  // rectangle length
  // public member functions
public:
  void setWidth (double );  // set rectangle width
  void setLength (double ); // set rectangle length
  double getWidth() const;  // get rectangle width
  double getLength() const; // get rectangle length
  double getArea() const;   // compute and get rectangle area
};

// Member function definitions
//*************************************************************
// setWidth assigns its argument to the private member width     *
//*************************************************************
void Rectangle::setWidth (double w)
{
  width = w;
}
//*************************************************************
// setLength assigns its argument to the private member length   *
//*************************************************************
void Rectangle::setLength (double l)
{
  length = l;
}
//*************************************************************
// getWidth returns the value in the private member width        *
//*************************************************************
double Rectangle::getWidth() const
{
  return width;
}
//*************************************************************
// getLength returns the value in the private member length      *
//*************************************************************
double Rectangle::getLength() const
{
  return length;
}
//*************************************************************
// getArea computes and returns the area from private data       *
//*************************************************************
double Rectangle::getArea() const
{
  return (width * length);
}

// main function
int main()
{
  Rectangle box1;
  box1.setWidth(12.7);
  box1.setLength(14.5);
  cout << "Box 1: "
       << box1.getWidth() << " x " << box1.getLength()
       << endl;
```

```
  Rectangle box2;
  box2.setWidth(8.5);
  box2.setLength(29.3);
  cout << "Box 2: "
       << box2.getWidth() << " x " << box2.getLength()
       << endl;
  // Test getArea()
  if (box1.getArea() < box2.getArea())
    cout << "Box 1 is smaller.\n";
  else
    cout << "Box 2 is smaller.\n";
  return 0;
}
```

```
Box 1: 12.7 x 14.5
Box 2: 8.5 x 29.3
Box 1 is smaller.
```

21. Separate interface and implementation files

- We separated `class` declaration (interface part) and member function definitions (implementation part) in an earlier exercise.
- Now, we're going to store the declaration in a header file `Rectangle.h`, the definitions in a C++ file `Rectangle.cpp`, and the testing program in a third file, `RectangleTest.cpp`.
- How can we run the test using these three files? Take a guess!
    1. The `Rectangle.h` file (which also contains the `#include` and `namespace` statements) will be included as a header file in both `Rectangle.cpp` and `RectangleTest.cpp`.
    2. `Rectangle.cpp` will be compiled into an object file `Rectangle.o` without linking using the `-c` option of the `g++` (GCC) compiler.
    3. `RectangleTest.cpp`, which contains the `main` function, will be compiled and linked with `Rectangle.o`.
    4. It turns out that the Linux `make` command can execute all of these steps using a configuration file called a `Makefile`.
- We're first going to do this manually on Google Cloud Shell, and then with a `Makefile`.

22. PRACTICE Tangle separate files

- Create `Rectangle.h`

```
#ifndef RECTANGLE_H
#define RECTANGLE_H
#include <iostream>
// Namespace declaration
using namespace std;

// Class declaration
class Rectangle {
  // private member data
private:
  double width; // rectangle width
  double length;  // rectangle length
  // public member functions
public:
```

```
  void setWidth (double );  // set rectangle width
  void setLength (double ); // set rectangle length
  double getWidth() const;  // get rectangle width
  double getLength() const; // get rectangle length
  double getArea() const;   // compute and get rectangle area
};
#endif
```

- Create `Rectangle.cpp`

```
#include "Rectangle.h"

// Member function definitions
//*************************************************************
// setWidth assigns its argument to the private member width     *
//*************************************************************
void Rectangle::setWidth (double w)
{
  width = w;
}
//*************************************************************
// setLength assigns its argument to the private member length   *
//*************************************************************
void Rectangle::setLength (double l)
{
  length = l;
}
//*************************************************************
// getWidth returns the value in the private member width        *
//*************************************************************
double Rectangle::getWidth() const
{
  return width;
}
//*************************************************************
// getLength returns the value in the private member length      *
//*************************************************************
double Rectangle::getLength() const
{
  return length;
}
//*************************************************************
// getArea computes and returns the area from private data       *
//*************************************************************
double Rectangle::getArea() const
{
  return (width * length);
}
```

- Create `RectangleTest.cpp`

```
#include "Rectangle.h"

// main function
int main()
{
  Rectangle box1;
  box1.setWidth(12.7);
  box1.setLength(14.5);
  cout << "Box 1: "
       << box1.getWidth() << " x " << box1.getLength()
```

```
        << endl;
  Rectangle box2;
  box2.setWidth(8.5);
  box2.setLength(29.3);
  cout << "Box 2: "
       << box2.getWidth() << " x " << box2.getLength()
       << endl;
  // Test getArea()
  if (box1.getArea() < box2.getArea())
    cout << "Box 1 is smaller.\n";
  else
    cout << "Box 2 is smaller.\n";
  return 0;
}
```

23. PRACTICE Compiling and testing a `class` from multiple files

1. Log into your Lyon Google account and open ide.cloud.google.com
2. Wait until your Linux Virtual Machine has been started (1 min).
3. Ignore the push to use Gemini, the AI assistant, instead just enlarge the terminal window (lower half of the screen).

4. The following commands are useful - enter them after the `$` prompt:

```
$ ll              # alias for `ls -alF` show details of all files
$ cd dir          # change to directory `dir`
$ cd ..           # go one level up
$ nano file       # open `file` in the `nano` editor
$ man cmd         # get documentation on `cmd`
$ g++ file -o out # compile `file` into object file `out`
$ file file       # give information about `file`
$ clear           # clear the screen
$ cat file        # display the content of `file`
$ rm file         # delete `file`
```

5. Get the three files from GitHub using `wget`. Enter the following commands at the `$` prompt, then check they're there with `ll`.

```
wget -O Rectangle.h tinuyrl.com/rectangle-h
wget -O Rectangle.cpp tinuyrl.com/rectangle-cpp
wget -O RectangleTest.cpp tinuyrl.com/rectangletest-cpp
```

6. Execute the following commands to build the executable `rect`, and then run it in the last step:

```
g++ -c Rectangle.cpp
g++ RectangleTest.cpp Rectangle.o -o rect
./rect
```

   If any of your .cpp or .h files cannot be found, add the current directory (`.`) to your `PATH` with: `export PATH="$PATH:."`

7. Use `wget` to get the `Makefile` from tinyurl.com/rectangle-makefile and store it as `Makefile`.

```
wget -O Makefile tinyurl.com/rectangle-makefile
```

8. Let's take a look at it to understand what it does:

```
rectangle: RectangleTest.o Rectangle.o
g++ RectangleTest.o Rectangle.o -o rectangle

Rectangle.o: Rectangle.cpp Rectangle.h
g++ -c Rectangle.cpp

RectangleTest.o: RectangleTest.cpp Rectangle.h
g++ -c RectangleTest.cpp

clean:
rm -f *.o rectangle
```

9. To use the `Makefile` with `make`, simply run `make` on the Google shell, and then run the executable `./rectangle` to check the outcome. If you run `make` again now, it will not do anything (no file changed):

```
g++ -c RectangleTest.cpp
g++ -c Rectangle.cpp
g++ RectangleTest.o Rectangle.o -o rectangle
```

10. To see `make`'s magic at work, enter the command `touch Rectangle.o` This will change its timestamp and `make` will get to work. Finally, you can run `make clean` to clean up and remove temporary files.

```
g++ RectangleTest.o Rectangle.o -o rectangle
```

24. PRACTICE Review questions

1. A header file that contains a class declaration is called
   - [X] a class specification file
   - [ ] a class implementation file
   - [ ] a class header file
   - [ ] a class definition file
2. The name of the class specification file is usually the same as the name of the class, with a .cpp extension (false)
3. What is stored in a separate .cpp file called the **class implementation** file?
   - [ ] static variable definitions
   - [ ] private members
   - [ ] public members
   - [ ] member function definitions
4. An include guard (preprocessor directive) prevents the header from accidentally being included more than once (true).
5. The standard library header files like `<iostream>` and the `namespace` declarations can be included more than once. (true)

25. PRACTICE Planning a house (multiple `Rectangle` objects)

- Constraints: Includes No description for this link (needs to be tangled and be available as `../src/Rectangle.cpp`, and `Rectangle.h` must also be there). There are two other versions of this application:
  1. One with dynamical allocation, and
  2. one with smart pointers to class objects.

- Sample input/output:

```
: What is the kitchen's length? 10
: What is the kitchen's width? 14
: What is the bedroom's length? 15
: What is the bedroom's width? 12
: What is the den's length? 20
: What is the den's width? 30
: The total area of the three rooms is 920
```

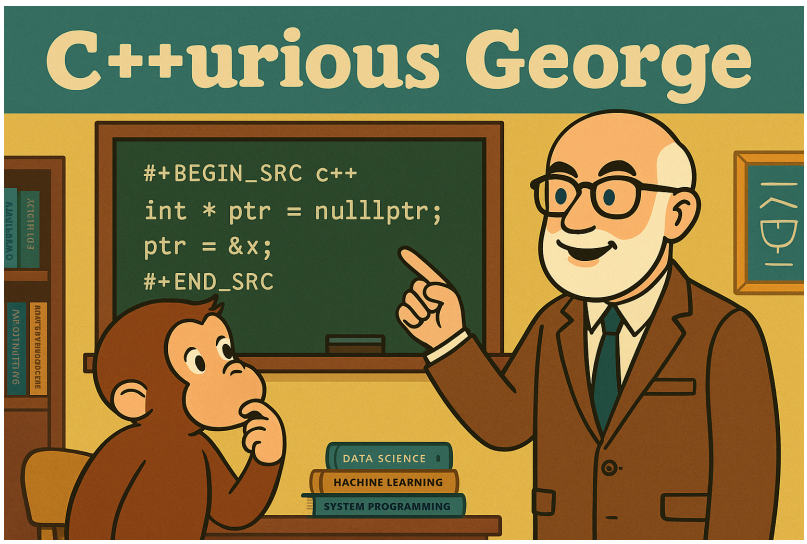
- Code:

```
#include <iostream>
#include "Rectangle.cpp"
using namespace std;

//*************************************************************
// Function main
//*************************************************************
int main()
{
  double number;     // number input
  double totalArea;  // total area
  Rectangle kitchen; // kitchen dimensions
  Rectangle bedroom; // bedroom dimensions
  Rectangle den;     // den dimensions

  // Get the kitchen dimensions
  cout << "What is the kitchen's length? ";
  cin >> number; cout << number << endl;    // get the length
  kitchen.setLength(number);                // store in kitchen object
  cout << "What is the kitchen's width? ";
  cin >> number; cout << number << endl;    // get the width
  kitchen.setWidth(number);                 // store in kitchen object

  // Get the bedroom dimensions
  cout << "What is the bedroom's length? ";
  cin >> number; cout << number << endl;    // get the length
  bedroom.setLength(number);                // store in bedroom object
  cout << "What is the bedroom's width? ";
  cin >> number;  cout << number << endl;   // get the width
  bedroom.setWidth(number);                 // store in bedroom object

  // Get the den dimensions
  cout << "What is the den's length? ";
  cin >> number; cout << number << endl;    // get the length
  den.setLength(number);                    // store in den object
  cout << "What is the den's width? ";
  cin >> number; cout << number << endl;    // get the width
  den.setWidth(number);                     // store in den object

  // Calculate the total area of the three rooms
  totalArea = kitchen.getArea() +
    bedroom.getArea() +
    den.getArea();

  // Display total area of the three rooms
  cout << "The total area of the three rooms is "
       << totalArea << endl;
  return 0;
}
```

- Testing the tangled program (org-babel-tangle) - cursor right after the command, then C-x C-e with the values

```
cd ../src
g++ rectangle2.cpp -o rectangle2
echo "10 14 15 12 20 30" | ./rectangle2
```

26. C++urious George: default constructor



- You can define a non-acting default constructor:

```
class X {
private:
  int a;
public:
  X() {}
};
int main()
{
  X x;
  return 0;
}
```

- Why can the standard library header files like `<iostream>` and the `namespace` declarations can be included more than once?

  Standard headers use include guards (or the directive `#pragma once`) internally so the compiler processes their contents only once even if you include them multiple times.

  `namespace` declarations can be repeated: They just tell the compiler to treat unqualified names as if they were in that namespace so re-declaring is harmless.

27. Avoiding Stale Data

- In No description for this link, the `getArea` member function returns the result of a calculation. Why don't we store the area in a member variable like `length` and `width`?

  > We do not store the area in a member variable because it may become **stale**: It depends wholly on two other member variables, and it would become incorrect as soon as one of these other variables changed.

- When designing a class do not store calculated data in member variables because they could become stale. Instead provide a member function that returns the result of the calculation.

28. Pointers to `class` objects

- You can define a pointer that points at a `class` object. You can then use the `->` operator to call member functions.

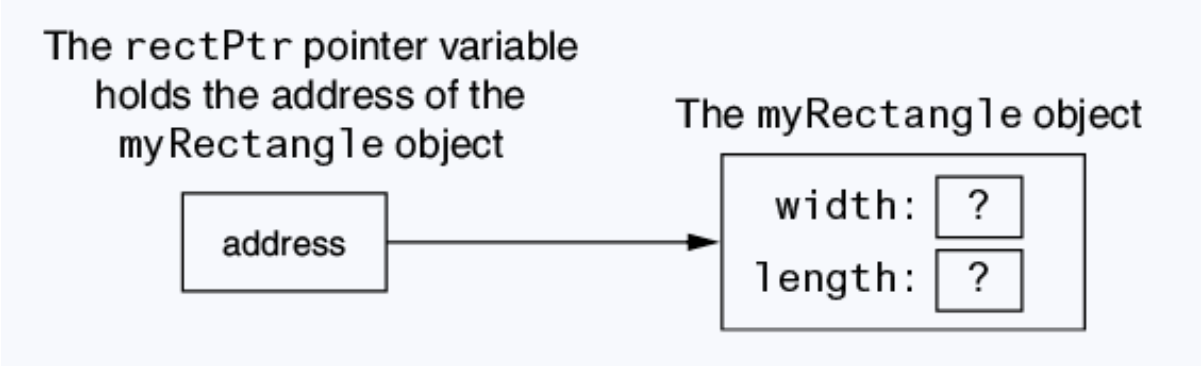
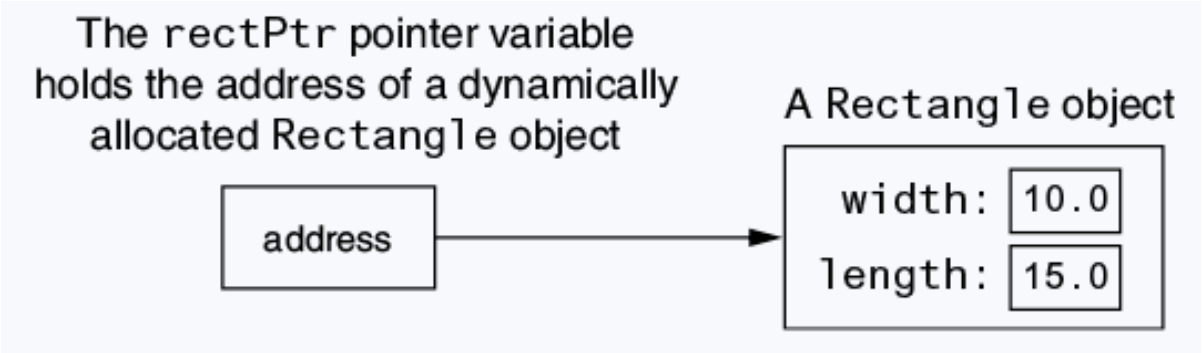


Figure 4: State of the object pointed at before initialization.

```
#include <iostream>
#include "Rectangle.cpp"
using namespace std;

int main()
{
  Rectangle myRectangle;          // create a rectangle
  Rectangle *rectPtr = nullptr;   // create a pointer
  rectPtr = &myRectangle;         // pointer points at rectangle

  rectPtr->setWidth(12.5);
  rectPtr->setLength(4.8);

  cout << "Width: " << rectPtr->getWidth() << endl;
  cout << "Length: " << rectPtr->getLength() << endl;
  return 0;
}
```

```
Width: 12.5
Length: 4.8
```

- Class object pointers can be used to dynamically allocate objects. Unless you use a smart pointer, you have to `delete` the object when you're done.




```
#include <iostream>
#include "Rectangle.cpp"
using namespace std;

int main()
{
  Rectangle myRectangle;          // create a rectangle
  Rectangle *rectPtr = nullptr;   // create a pointer
  rectPtr = new Rectangle;        // dynamically allocate a Rectangle object

  rectPtr->setWidth(10.0);        // store width value
  rectPtr->setLength(15.0);       // store length value

  cout << "Width: " << rectPtr->getWidth() << endl;
  cout << "Length: " << rectPtr->getLength() << endl;

  delete rectPtr;                 // free the allocated memory
  rectPtr = nullptr;              // store address 0 in pointer

  return 0;
}
```

```
Width: 10
Length: 15
```

29. PRACTICE Using pointers to objects [Bonus assignment]

- **Task:** Modify No description for this link to dynamically allocate `Rectangle` objects, `kitchen`, `bedroom`, and `den`.
- **Constraints:** Includes No description for this link (needs to be tangled and be available as `../src/Rectangle.cpp`, and `Rectangle.h` must also be there).

- **Sample input/output:**

```
: What is the kitchen's length? 10
: What is the kitchen's width? 14
: What is the bedroom's length? 15
: What is the bedroom's width? 12
: What is the den's length? 20
```

```
: What is the den's width? 30
: The total area of the three rooms is 920
```

- **Solution:**

```
#include <iostream>
#include "Rectangle.cpp"
using namespace std;

//***************************************************************
// Function main
//***************************************************************
int main()
{
  double number;                    // number input
  double totalArea;                 // total area
  Rectangle *kitchen = nullptr;     // pointer to Rectangle object
  kitchen = new Rectangle;          // kitchen dimensions
  Rectangle *bedroom = nullptr;     // pointer to Rectangle object
  bedroom = new Rectangle;          // bedroom dimensions
  Rectangle *den = nullptr;         // pointer to Rectangle object
  den = new Rectangle;              // den dimensions

  // Get the kitchen dimensions
  cout << "What is the kitchen's length? ";
  cin >> number; cout << number << endl;     // get the length
  kitchen->setLength(number);                // store in kitchen object
  cout << "What is the kitchen's width? ";
  cin >> number; cout << number << endl;     // get the width
  kitchen->setWidth(number);                 // store in kitchen object

  // Get the bedroom dimensions
  cout << "What is the bedroom's length? ";
  cin >> number; cout << number << endl;     // get the length
  bedroom->setLength(number);                // store in bedroom object
  cout << "What is the bedroom's width? ";
  cin >> number;  cout << number << endl;    // get the width
  bedroom->setWidth(number);                 // store in bedroom object

  // Get the den dimensions
  cout << "What is the den's length? ";
  cin >> number; cout << number << endl;    // get the length
  den->setLength(number);                   // store in den object
  cout << "What is the den's width? ";
  cin >> number; cout << number << endl;    // get the width
  den->setWidth(number);                    // store in den object

  // Calculate the total area of the three rooms
  totalArea = kitchen->getArea() +
    bedroom->getArea() +
    den->getArea();

  // Display total area of the three rooms
  cout << "The total area of the three rooms is "
       << totalArea << endl;

  // Deallocate memory
  delete kitchen;
  kitchen = nullptr;
  delete bedroom;
  bedroom = nullptr;
  delete den;
  den = nullptr;
```

```
  return 0;
}
```

- Testing the tangled program (org-babel-tangle) - cursor right after the command, then C-x C-e with the values

```
cd ../src
g++ rectangle3.cpp -o rectangle3
echo "10 14 15 12 20 30" | ./rectangle3
```

30. SOMEDAY Using Smart Pointers to Allocate Objects

- You can use smart pointers to automatically delete a chunk of dynamically allocated memory when it is no longer being used.
- Requires `#include <memory>`

- Code Example:

1. Using No description for this link,
2. modifying No description for this link.

```
#include <iostream>
#include <memory>
#include "Rectangle.cpp"
using namespace std;

//***************************************************************
// Function main
//***************************************************************
int main()
{
  double number;                    // number input
  double totalArea;                 // total area

  // Dynamically allocate the objects.
  unique_ptr<Rectangle> kitchen(new Rectangle);
  unique_ptr<Rectangle> bedroom(new Rectangle);
  unique_ptr<Rectangle> den(new Rectangle);

  // Get the kitchen dimensions
  cout << "What is the kitchen's length? ";
  cin >> number; cout << number << endl;     // get the length
  kitchen->setLength(number);                // store in kitchen object
  cout << "What is the kitchen's width? ";
  cin >> number; cout << number << endl;     // get the width
  kitchen->setWidth(number);                 // store in kitchen object

  // Get the bedroom dimensions
  cout << "What is the bedroom's length? ";
  cin >> number; cout << number << endl;     // get the length
  bedroom->setLength(number);                // store in bedroom object
  cout << "What is the bedroom's width? ";
  cin >> number;  cout << number << endl;    // get the width
  bedroom->setWidth(number);                 // store in bedroom object

  // Get the den dimensions
  cout << "What is the den's length? ";
  cin >> number; cout << number << endl;    // get the length
```

```
  den->setLength(number);                    // store in den object
  cout << "What is the den's width? ";
  cin >> number; cout << number << endl;     // get the width
  den->setWidth(number);                     // store in den object

  // Calculate the total area of the three rooms
  totalArea = kitchen->getArea() +
    bedroom->getArea() +
    den->getArea();

  // Display total area of the three rooms
  cout << "The total area of the three rooms is "
       << totalArea << endl;

  return 0;
}
```

- Testing the tangled program (org-babel-tangle) - cursor right after the command, then C-x C-e with the values

```
cd ../src
g++ rectangle4.cpp -o rectangle4
echo "10 14 15 12 20 30" | ./rectangle4
```

31. PRACTICE Review Questions

1. What is defining a `class` object called?
   - [X] Instantiation
   - [ ] Allocation
   - [ ] Declaration
   - [ ] Definition
2. When is a variable in danger of becoming stale and what to do about it? Example?
3. You can use the `new` operator to dynamically allocate an instance of a class. Give an example!

4. Given the following code, which of the statements below correctly calls the `setRadius` member function passing 2.5 as an argument?

```
Circle *circlePtr = nullptr;
circlePtr = new Circle;
```

   - [ ] `nullptr.setRadius(2.5);`
   - [X] `circlePtr->setRadius(2.5);`
   - [ ] `Circle::setRadius(2.5);`
   - [ ] `circlePtr.setRadius(2.5);`
5. You must declare all `private` members of a `class` before the `public` members (false).
6. Assume `RetailItem` is the name of a `class`, with a `void` member function named `setPrice`, which accepts a `double` argument. Which of the following shows the correct use of the scope resolution operator in the member function definition?
   - [ ] `RetailItem::void setPrice(double p);`
   - [X] `void RetailItem::setPrice(double p);`
7. An object's `private` member variables are accessed from outside the object by which of the following?
   - [X] `public` member functions
   - [ ] any function
   - [ ] the dot operator

    - [ ] the scope resolution operator
8. Assume `RetailItem` is the name of a `class` with a `void` member function named `setPrice`, which accepts a `double` argument. If `soap` is an instance of the `RetailItem` `class`, which of the following statements properly uses the `soap` object to call the `setPrice` member function?
    - [ ] `RetailItem::setPrice(1.49);`
    - [ ] `soap::setPrice(1.49);`
    - [X] `soap.setPrice(1.49);`
    - [ ] `soap:setPrice(1.49);`
9. Assume `RetailItem` is the name of a `class` with a `void` member function named `setPrice`, which accepts a `double` argument. If `soap` is a dynamically allocated pointer to a `RetailItem` object, which of the following statements properly uses the `soap` pointer to call the `setPrice` member function?
    - [ ] `RetailItem::setPrice(1.49);`
    - [ ] `soap.setPrice(1.49);`
    - [X] `soap->setPrice(1.49);`
    - [ ] `soap:setPrice(1.49);`

10. Complete the following code skeleton to declare a `class` named `Date`. The class should contain variables and functions to store and retrieve a date in the form 4/2/2021.

```
class Date
{
private:
  int month;
  int day;
  int year;
public:
  Date(int m, int d, int y);
  void setMonth(int m);
  void setDay(int d);
  void setYear(int y);
  int getMonth();
  int getDay();
  int getYear();
};
```

32. SOMEDAY Why Have Private Members?

- In OOP, an object protects its important data by making them `private`, and provides a `public` interface to access the data.
- Member functions can protect against invalid data values and you can use `exit(EXIT_FAILURE)` from `<cstdlib>` to leave the application.
- There is a whole methodology of implementing invalid data protections called **exceptions**.

33. SOMEDAY Inline member functions

- Inline functions are member functions whose body is written inside a class declaration.
- Inline functions are compiled differently from other functions: They are not called but expanded inline the call is replaced with the function code itself, which results in a performance improvement.
- However, the program size therefore necessarily increases if the function is called multiple times.

34. Review Questions

1. Why would you declare a `class` member variable `private`?

You would declare data `private` to enforce encapsulation, keeping the data hidden from outside code.

2. When a `class` member variable is declared `private`, how does code outside the class store values in, or retrieve values from, the member variables?

   `private` data are changed by mutator (setter), and retrieved by accessor (getter) `public` member functions.

3. What is a class specification file? What is a class implementation file?

   The `class` specification file (class.h) contains the `class` declaration and prototypes (signatures) of `public` member functions and the `private` member data.

   The `class` implementation file (.cpp) contains the `class` member function definitions, identified as belonging to the class by the class name followed by the scope resolution operator before the function name.

4. What is the purpose of an include guard?

   An include guard starting with `#ifndef` protects class specifications from being included multiple times.

5. Assume the following class components exist in a program:

`BasePay` `class` declaration	BasePay.h
`BasePay` member function definition	BasePay.cpp
`Overtime` `class` declaration	Overtime.h
`Overtime` member function definitions	Overtime.cpp

   In what files would you store each of these components?

6. What is an inline member function? What's the effect of using an inline member function instead of a regular (external) definition?

35. Constructors

- What is a constructor?

  A constructor is a member function that is automatically called when a `class` object is created. It has the same name as the `class` and it does not have a `return` type.

- Example:

```
#include <iostream>
using namespace std;

class Demo
{
public:
  Demo(); // Constructor
};
```

```
Demo::Demo()
{
  cout << "Welcome to the Demo constructor!\n";
}

int main()
{
  Demo hello;
  return 0;
}
```

```
Welcome to the Demo constructor!
```

36. PRACTICE Improving `Rectangle` with a constructor

- We're going to make two improvements:
  1. Make the code safer against invalid data entry (implementation).
  2. Provide a constructor that initializes a rectangle of zero width and length using an initializer list.
- We'll keep the specification and the implementation apart in a .h and a .cpp file, and we'll test with a Makefile on the shell.

- `Rectangle.h` (with `#include` guard) - tangles but does not evaluate.

```
// Specification file for the Rectangle class
#ifndef RECTANGLE_H
#define RECTANGLE_H
class Rectangle
{
private:
  double width;
  double length;
public:
  Rectangle();                  // constructor
  void setWidth(double);        // mutator: Width
  void setLength(double);       // mutator: Length
  double getWidth() const       // accessor: Width
  {return width;}
  double getLength() const      // accessor: Length
  {return length;}
  double getArea() const        // accessor: Area
  {return width * length;}
};
#endif
```

- `Rectangle.cpp` (with `#include "Rectangle.h"`) - tangles but does not evaluate.

```
#include "Rectangle.h"
#include <iostream>
#include <cstdlib>
using namespace std;

//*************************************************************
// The constructor initializes width and length to 0.0
//*************************************************************
Rectangle::Rectangle() :
  width(0.0), length(0.0) {}
//*************************************************************
```

```
// setWidth sets the value of the member variable width
//***************************************************************
void Rectangle::setWidth(double w)
{
  if (w >= 0)
    width = w;
  else
    {
      cout << "Invalid width.\n";
      exit(EXIT_FAILURE);
    }
}
//***************************************************************
// setLength sets the value of the member variable length
//***************************************************************
void Rectangle::setLength(double l)
{
  if (l >= 0)
    length = l;
  else
    {
      cout << "Invalid length.\n";
      exit(EXIT_FAILURE);
    }
}
```

- `RectangleTest.cpp` (with `#include "Rectangle.h"`) - tangles but does not evaluate.

```
#include <iostream>
#include "Rectangle.h"
using namespace std;

int main()
{
  Rectangle box;
  // Display rectangle data
  cout << "Here is the rectangle's data:\n";
  cout << "Width: "  << box.getWidth()  << endl
       << "Length: " << box.getLength() << endl
       << "Area: "   << box.getArea()    << endl;

  return 0;
}
```

- In class (without Makefile): Test the new setup on the command-line:

```
cd ../src
g++ -c Rectangle.cpp -o Rectangle.o # produce Rectangle.o
g++ RectangleTest.cpp Rectangle.o -o rect # produce executable
./rect
```

- `Makefile` with targets for the executable `rectangle`, and the object file with our definitions (from `Rectangle.cpp`), `Rectangle.o`, and a `PHONY` target to `clean` up afterwards.

```
rectangle6: Rectangle.o Rectangle.h
g++ RectangleTest.cpp Rectangle.o -o rectangle6

Rectangle.o: Rectangle.cpp Rectangle.h
g++ -c Rectangle.cpp -o Rectangle.o
```

```
clean:
rm *.o rectangle6
```

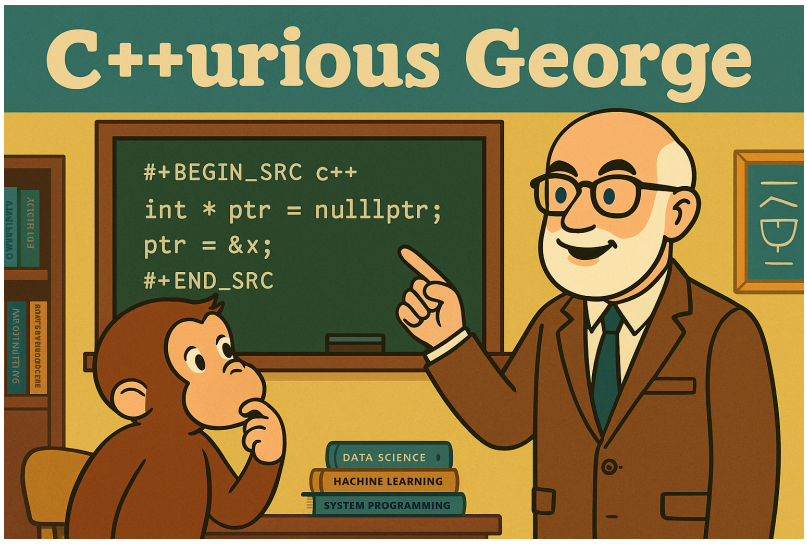
- Testing: Running `make` with the `Makefile` on the shell.

```
cd ../src
make rectangle6
./rectangle6
make clean
```

- From C++11, you can also initialize a member variable in-place, namely when the variables are defined:

```
class Rectangle
{
  private:
    double width = 0.0;
    double length = 0.0;
};
```

37. C++urious George: RAII, object lifecycle



- What is the RAII principle?

  "Resource Allocation Is Initialization:" Introduced by Stroustrup with C++. The constructor initializes resources (memory, file handles, etc.), and the destructor releases them when the object has reached the end of its lifetime (when it goes out of scope).

- Why did the `Rectangle` class (in No description for this link) not need a constructor?

  In C++, if you don't declare a constructor, the compiler provides a **default constructor** that initializes built-in types to arbitrary values and calls default constructors for any member objects.

The `Rectangle` class should have a constructor, otherwise e.g. calling the member function `getArea()` before setting `length` and `width` would return garbage results.

- Why are the `Rectangle` mutator functions defined in the implementation file but the `const` accessor functions are defined inline in the specification file?

    - Small simple and `const` member functions can be defined inline in the header file (.h) without performance cost.
    - This avoids the function call overhead for trivial operations.
    - Larger functions are moved to the implementation (.cpp) file to improve readability and avoid recompilation.

38. Passing Arguments to Constructors

- A constructor can have parameters and can accept arguments when an object is created.
- Example: Here is a constructor with parameters for the `Rectangle` class.

    1. `Rectangle.h`

    ```
    Rectangle(double,double);
    ```

    2. `Rectangle.cpp`

    ```
    Rectangle::Rectangle(double w, double l)
    {
       width = w;
       length = l;
    }
    ```

    Or with an initializer list:

    ```
    Rectangle::Rectangle(double w, double l) :
      width(w), length(l);
    ```

    3. `RectangleTest.cpp`:

    ```
    Rectangle box(10.0, 12.0); // creates 10 x 12 rectangle
    ```

- You can also pass **default arguments** to the constructor. In the example, a default tax `rate` of 5% is written into the constructor.

    In `Sale.h` for a `Sale` class:

    ```
    Sale(double cost, double rate=0.05); // constructor
    ```

    In `main`:

    ```
    Sale itemSale(cost); // The Sale item has rate = 0.05
    ```

- If a constructor has default arguments for all its parameters, it can be called without explicit arguments. It then becomes the default constructor.
- When all constructors of a `class` require arguments, there is no default constructor. If you don't pass arguments, you get an error.

39. PRACTICE `Rectangle class` with parameter constructor [TO DO]

- **Task**: Update and test the `Rectangle` class with a constructor that takes `width` and `length` as `double` parameters.

    1. Get the width and length of the house in feet from the user.
    2. Create a `Rectangle` object `house` with these measures.
    3. Display `house`'s `width`, `length`, and `area`.

```

```

40. Home assignment: `Sale` class

Solution in Gaddis, p. 795.

41. The Default Constructor and the RAII object lifecycle

- A **destructor** is a member function that is automatically called when an object is destroyed.
- When is an object destroyed?

    > Objects are destroyed when they go out of scope - for example at the end of a function if the objects are local - or when they are deleted with `delete` (or `delete []`) after being dynamically allocated.

- A demo example:

```
#include <iostream>
using namespace std;

// class interface
class Demo
{
public:
  Demo();
  ~Demo();
};

// class implementation
Demo::Demo() { cout << "Welcome to the constructor!\n"; }
Demo::~Demo() { cout << "The destructor is now running.\n"; }

// main function
// demonstrates an object wxith a constructor and a destructor
int main()
{
  Demo demoObject;
  return 0;
}
```

```
Welcome to the constructor!
The destructor is now running.
```


```
Welcome to the constructor!
The destructor is now running.
```


- What is the RAII principle?

  > "Resource Allocation Is Initialization:" Introduced by Stroustrup with C++. The constructor initializes resources (memory, file handles, etc.), and the destructor releases them when the object has reached the end of its lifetime (when it goes out of scope).

42. PRACTICE Destructors

- The extended example uses the `ContactInfo class`, which holds `name` and `phone` data about a contact.
- This program also serves as an introduction to some `string` functions. Since names and hone numbers may not have the same length everywhere, they are allocated dynamically. All member functions are defined inline.

- `ContactInfo.h`:

```
#ifndef CONTACTINFO_H
#define CONTACTINFO_H
#include <cstring>    // needed for strlen and strcpy

// ContactInfo interface
class ContactInfo
{
private:
  char *name;
  char *phone;
public:
  ContactInfo(const char *n, const char *p) // constructor
  {
    name = new char[strlen(n)+1]; // allocate memory for name
    phone = new char[strlen(p)+1]; // allocate memory for phone
    strcpy(name, n); // copy name to allocated memory
    strcpy(phone, p); // copy phone to allocated memory
  }
  ~ContactInfo()
  {
    delete [] name;
    delete [] phone;
  }
  const char *getName() const { return name; }
  const char *getPhoneNumber() const { return phone; }
};
#endif
```

- Notice that the getter functions return a pointer to a `const char` - this prevents code from changing the string that the pointer points to. This is useful for pointers and references.

- Testing the `ContactInfo` class after (org-babel-tangle):

```
#include <iostream>
#include "ContactInfo.h"
using namespace std;

int main()
{
  ContactInfo entry("Kristen Lee", "555-2021");

  cout << "Name: "         << entry.getName()        << endl
       << "Phone Number: " << entry.getPhoneNumber() << endl;

  return 0;
}
```

43. PRACTICE From C-style strings to the `string` class

- No description for this link code uses C-style strings. Change this to C++ style strings!
- Solution:

  Header file:

```
#ifndef CONTACTINFO2_H
#define CONTACTINFO2_H
#include <string>

// ContactInfo interface
class ContactInfo
{
private:
  std::string name;
  std::string phone;
public:
  ContactInfo(const std::string& n,
              const std::string& p) :
    name(n),phone(p){}
  const std::string& getName() const { return name; }
  const std::string& getPhoneNumber() const { return phone; }
};
#endif
```

  Test file: Uses `ContactInfo2.h`, that's the only difference.

```
#include <iostream>
#include "ContactInfo2.h"
using namespace std;

int main()
{
  ContactInfo entry("Kristen Lee", "555-2021");

  cout << "Name: "         << entry.getName()        << endl
       << "Phone Number: " << entry.getPhoneNumber() << endl;

  return 0;
}
```

- Notice that using reference parameters (`string&`) avoids unnecessary copies.

44. Overloading Constructors

- A class can have more than one constructor. Just like with functions, the compiler will tell them apart as long as the parameter lists are different.
- Example: The `string` class (doc).

```
string str; // default constructor
string str("Hello"); // overloaded constructor
```

- One constructor can call another constructor in the same class ("constructor delegation"). In the example, the default constructor `Contact()` calls another constructor to initialize the object:

```
class Contact
{
private:
  string name;
  string email;
  string phone;
public:
  Contact() : Contact("","","") // initialize Contact with empty chars
  {}
  Contact(string n, string e, string p) :
    name(n), email(e), phone(p)
  {}
};
```

- There can be only one default constructor and one destructor: If there was more than one constructor without parameters, or more than one destructor (without arguments by design) the compiler would not know which one to execute.
- Member functions other than constructors can also be overloaded. This can be useful if you want to define different ways to perform the same operation.
- Example: The second `setCost` function could be used if `cost` is stored in a `string` object. The function `stod` converts a `string` to a `double`.

```
void setCost(double c) { cost = c; }
void setCost(string s) { cost = stod(s); }
```

45. PRACTICE Overloading constructors

- Write an interface for a `class InventoryItem`. The member data are a description (`string`), the item `cost` (`double`), and the number of `units` on hand (`int`). Define all member functions inline and store the `class` interface in `InventoryItem.h` with an include guard.
- The class should have three constructors:
  1. A default constructor with an empty description, and zero values for the numeric data.
  2. A constructor that accepts a description as an argument and initializes `cost` and `units` to zero.
  3. A constructor that accepts all data members as arguments
- Mutators: `setDescription`, `setCost`, `setUnits`
- Accessors: `getDescription`, `getCost`, `getUnits`

- Sample output:

```
The following items are in inventory:

Description: Hammer
Cost: $6.95
Units: 12

Description: Pliers
Cost: $0
Units: 0

Description: Wrench
Cost: $8.75
Units: 20
```

- Solution:

  InventoryItem.h:

```
#ifndef INVENTORYITEM_H
#define INVENTORYITEM_H
#include <string>
using std::string;

class InventoryItem
{
private:
  string description;
  double cost;
  int units;
public:

  // Constructor #1
  InventoryItem()
  {
    description = "";
    cost = 0.0;
    units = 0;
  }
  // Constructor #2
  InventoryItem(string d)
  {
    description = d;
    cost = 0.0;
    units = 0;
  }

  // Constructor #3
  InventoryItem(string d, double c, int u)
  {
    description = d;
    cost = c;
    units = u;
  }

  // Mutators
  void setDescription(string d) { description = d; }
  void setCost(double c) { cost = c; }
  void setUnits(int u) { units = u; }
  // Accessors
  string getDescription() { return description; }
```

```
  double getCost() { return cost; }
  int getUnits() { return units; }
};
#endif
```

- Testing `main` function:

```
#include <iostream>
#include "InventoryItem.h"
using std::cout;

int main()
{
  InventoryItem item1; // constructor #1
  item1.setDescription("Hammer");
  item1.setCost(6.95);
  item1.setUnits(12);

  InventoryItem item2("Pliers"); // constructor #2

  InventoryItem item3("Wrench",8.75,20); // constructor #3

  // Display inventory data
  cout << "The following items are in inventory:\n\n";

  cout << "Description: " << item1.getDescription() << '\n'
       << "Cost: $"       << item1.getCost()        << '\n'
       << "Units: "       << item1.getUnits()       << "\n\n";

  cout << "Description: " << item2.getDescription() << '\n'
       << "Cost: $"       << item2.getCost()        << '\n'
       << "Units: "       << item2.getUnits()       << "\n\n";

  cout << "Description: " << item3.getDescription() << '\n'
       << "Cost: $"       << item3.getCost()        << '\n'
       << "Units: "       << item3.getUnits()       << '\n';

  return 0;
}
```

46. PRACTICE Review Questions

1. Constructors may have default arguments. (true)
2. All member functions can be initialized with lists (false)

3. In the following code fragment, what is the value of `age` for `voter2`?

```
#include <string>
#include <iostream>
using namespace std;

class Person
{
private:
  string name;
  int age;
public:
  Person(string n, int a = 18) :
    name(n), age(a) {}
```

```
  void setName(string n) {name=n;}
  void setAge(int a) {age=a;}
  string getName() const {return name;}
  int getAge() const {return age;}
};

int main()
{
  Person voter1 ("Lyndon Taft", 47);
  Person voter2 ("Timothy Burke");

  cout << "-- Information for voter #1 --\n"
       << "Name: " << voter1.getName() << endl
       << "Age: "  << voter1.getAge()  << endl;

  cout << "-- Information for voter # --\n"
       << "Name: " << voter2.getName() << endl
       << "Age: "  << voter2.getAge()  << endl;

  return 0;
}
```

```
-- Information for voter #1 --
Name: Lyndon Taft
Age: 47
-- Information for voter # --
Name: Timothy Burke
Age: 18
```

- [ ] -1
- [ ] 0
- [X] 18
- [ ] 47

4. A constructor whose arguments all have default values is a default constructor. (true)
5. When all constructors of a class require arguments, then the class does not have a default constructor. (true)
6. Destructors are never called with a `return` type. (true)
7. Destructors may take any number of arguments. (false)
8. Which of these leads to automatic deletion of an object by calling a destructor when the object goes out of scope?
   - [ ] String literal
   - [ ] Pointer to a C-string
   - [ ] `new` operator
   - [X] Smart pointer
9. Briefly describe the purpose of a constructor.
10. Briefly describe the purpose of a destructor.
11. Which of these is a member function that is never declared with a `return` data type but that may have arguments:
    - [X] The constructor
    - [ ] The destructor
    - [ ] Both the constructor and the destructor
    - [ ] Neither the constructor nor the destructor
12. Which of these is a member function that is never declared with a `return` data type and that can never have arguments:

- [ ] The constructor
- [X] The destructor
- [ ] Both the constructor and the destructor
- [ ] Neither the constructor nor the destructor

13. Destructor function names always start with a:
    - [X] Tilde character ~
    - [ ] A number
    - [ ] A data type name
    - [ ] None of the above
14. A constructor that requires no arguments is called:
    - [X] A default constructor
    - [ ] An overloaded constructor
    - [ ] A null constructor
    - [ ] None of the above
15. Which of these `class` member functions returns a pointer to a `const char` and does not change the value of its argument?
    - [X] `const char* Person(const char *) const;`
    - [ ] `char* Person(const char *);`
    - [ ] `const char* Person(const char *);`
    - [ ] None of the above
16. A class may have more than one constructor, as long as each has a different
    - [ ] return type
    - [X] list of parameters
    - [ ] instance
    - [ ] name

17. It would be an error to create a constructor that accepts no parameters along with another constructor that has default arguments for all its parameters. (true)

    This is `true` because a constructor that has default parameters for all its arguments can be called without arguments and therefore cannot be distinguished from the default constructor.

18. Classes may have only one destructor. (true)
19. Member functions other than the constructor cannot be overloaded. (false)
20. A constructor can call another constructor in the same class. (true)

21. What will the following program display on the screen?

```
#include <iostream>
using namespace std;

class Tank
{
private:
   int gallons;
public:
   Tank()
      { gallons = 50; }
   Tank(int gal)
      { gallons = gal; }
   int getGallons()
      { return gallons; }
};
int main()
{
   Tank storage[3] = { 10, 20 };
   for (int index = 0; index < 3; index++)
      cout << storage[index].getGallons() << endl;
   return 0;
}
```

22. What will the following program display on the screen?

```
#include <iostream>
using namespace std;

class Package
{
private:
   int value;
public:
   Package()
      { value = 7; cout << value << endl; }
   Package(int v)
      { value = v; cout << value << endl; }
   ~Package()
      { cout << value << endl; }
};

int main()
{
   Package obj1(4);
   Package obj2;
   Package obj3(2);
   return 0;
}
```

47. SOMEDAY Private Member Functions

48. SOMEDAY Arrays of Objects

With animation/walk-through

Defines an array of `InventoryItem` objects:

```
const int ARRAY_SIZE = 40;
InventoryItem inventory[ARRAY_SIZE];
```

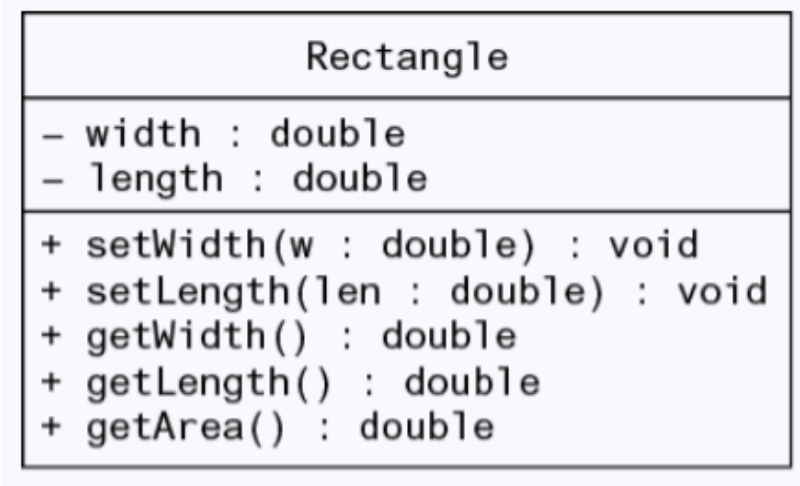
49. SOMEDAY Review questions

Page 812

50. PRACTICE OOP Case Study: Bank `Account` [turn this into a lab?]

51. PRACTICE OOP Case Study: Simulating Dice with `Die` [turn this into a lab?]

52. SOMEDAY Object-Oriented Design with UML



53. Programming Challenges

54. SOMEDAY Instance and static members

Each instance of a `class` has its own copies of the `class` instance variables. If a member variable is declared `static`, however, all instances of that `class` have access to that variable. If a member function is declared `static`, it may be called without any instances of the `class` being defined.

55. SOMEDAY Friends of classes

A `friend` is a function or `class` that is not a member of a `class`, but has access to the private members of the `class`.

Author: Marcus Birkenkrahe
Created: 2026-04-25 Sat 08:18